PROGRAM :	***SMART COMPUTING***
SUBMITTED TO :	***PROFESSOR SARWAR***
NAMES	***ID***
MUHAMMAD HAMZA SHAKEEL	***2417295***
ROSHAN AHMAD	***2417296***
HUZAIFA AHMAD QURESHI	***2417297***
WALEED SATTAR	***2417301***
NIHARIKA THAPA MAGAR	***2417135***

→DEPARTMENT OF SMART COMPUTING←

PROJECT PROPOSAL__RESEARCH METHADOLOGY

→*PROJECT PROPOSAL* ←

AI BASED DOCUMENT SIMPLIFICATION AND ACTION RECOMMENDED

ABBREVIATIONS

AI	**Artificial Intelligence**
OCR	**Optical Character Recognition**
NLP	**Natural Language Processing**
API	**Application Programming Interface**
UI	**User Interface**
PDF	**Portable Document Format**
ML	**Machine Learning**
DB	**Database**
HCI	**Human-Computer Interaction**
SRS	**Software Requirements Specification**
SDLC	**Software Development Life Cycle**

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Brief 1
1.2 Relevance to Course Modules 1
1.3 Project Background 2
1.4 Literature Review 2
1.5 Analysis from Literature Review (in the context of your project) 3
1.6 Methodology and Software Lifecycle for This Project 3
1.6.1 The rationale behind Selected Methodology 4

2 PROBLEM DEFINITION 5

2.1 Problem Statement 5
2.2 Deliverables and Development Requirements 6
2.3 Current System (if applicable to your project) 6

3 REQUIREMENT ANALYSIS 7

3.1 Use Cases Diagram(s) 7
3.2 Detailed Use Case 8
3.3 Functional Requirements 9
3.4 Non-Functional Requirements 10

4 DESIGN AND ARCHITECTURE 11

4.1 System Architecture 11
4.2 Data Representation [Diagram + Description] 12
4.3 Process Flow/Representation 13
4.4 Design Models [along with descriptions] 14

5 IMPLEMENTATION 15

5.1 Algorithm 15
5.2 External APIs 16
5.3 User Interface 17

6 TESTING AND EVALUATION 18

6.1 Manual Testing 18
6.1.1 System testing 19
6.1.2 Unit Testing 20
6.1.3 Functional Testing 21
6.1.4 Integration Testing 22
6.2 Automated Testing 23
Tools used: 24

7 CONCLUSION AND FUTURE WORK 25

7.1 Conclusion 25
7.2 Future Work 26

8 REFERENCES 27

LIST OF FIGURES

Figure 1.1 System Overview Diagram ..... ……..……..……..

LIST OF TABLES

Table 2.1 Detailed Use Case 8

Table 3.1 Functional Requirements 9

Table 3.2 Non-Functional Requirements 10

Table 4.1 Data Representation 12

Table 5.1 External APIs Used 16

Table 6.1 Unit Testing 20

Table 6.2 Functional Testing 21

Table 6.3 Integration Testing 22

1. INTRODUCTION

1.1 Brief

This project presents an AI-based document understanding and action recommendation system designed for foreign students in Korea. The system processes official documents such as visa notices, health insurance bills, tax documents, and university letters. Using Optical Character Recognition (OCR) and Natural Language Processing (NLP), the system extracts, analyzes, and simplifies document content. It provides a clear summary, identifies important details such as deadlines and payment amounts, and suggests actions to the user.

1.2 Relevance to Course Modules

This project integrates multiple computing concepts:

- Artificial Intelligence (NLP and classification)
- Software Engineering (system design and development lifecycle)
- Database Systems (data storage and retrieval)
- Human-Computer Interaction (user interface design)
- Web Development (frontend and backend integration)

1.3 Project Background

Foreign students in Korea frequently receive official documents written in Korean. Due to language barriers and complex formatting, these documents are difficult to understand. Misinterpretation often leads to missed deadlines, financial penalties, or legal complications. Existing tools only translate or extract text but fail to provide meaningful understanding or guidance.

1.4 Literature Review

Existing systems include OCR tools, translation services, and document summarization platforms. These systems can extract text and convert it into another language. However, they do not identify important elements such as deadlines, required actions, or risks. Most tools provide passive information rather than helping users make decisions.

Several existing systems provide document processing and analysis capabilities using AI techniques. However, they are limited in providing actionable insights.

docAnalyzer.ai is an AI-powered platform that allows users to upload documents and interact with them through natural language queries. It performs OCR and semantic analysis effectively, enabling users to extract insights from documents. However, it lacks

structured action recommendations and does not focus on identifying deadlines or risks in official documents.

DocSimplify is a document analysis tool that provides summarization and data extraction features. It helps users understand large documents by generating concise summaries and highlighting key information. Despite this, it does not provide guidance on required actions or alert users about important deadlines.

Linnk AI offers multi-language document processing, including translation, OCR, and summarization. It is useful for understanding content in different languages. However, it does not interpret legal or official documents and does not provide action-based outputs or risk analysis.

ABBYY FineReader is a widely used OCR tool known for its high accuracy in text extraction from scanned documents. It is mainly used for document digitization. However, it lacks intelligence in understanding document meaning and does not provide any decision-support or user guidance.

pdfFiller is an AI-based document management and editing tool that supports summarization, translation, and document interaction. While it allows users to manipulate documents efficiently, it focuses more on editing rather than understanding and does not provide structured recommendations or alerts.

1.5 Analysis from Literature Review

From the above analysis, it is clear that existing systems primarily focus on text extraction, translation, and summarization. However, they lack the ability to interpret documents and provide actionable guidance. The proposed system addresses this gap by identifying key information such as deadlines and required actions, and presenting them in a clear and user-friendly manner.

1.6 Methodology and Software Lifecycle

The project follows the Agile Software Development Model. Development is carried out in phases including planning, implementation, testing, and refinement. Each module is developed and tested iteratively.

1.6.1 Rationale for Methodology

Agile is selected because it allows flexibility, continuous improvement, and better handling of changing requirements.

2. PROBLEM DEFINITION

2.1 Problem Statement

Foreign students in Korea face difficulty understanding official documents due to language barriers and complex terminology. This leads to confusion, missed deadlines, financial penalties, and stress.

2.2 Deliverables and Development Requirements

- Web-based application
- OCR module for text extraction
- NLP module for analysis
- Dashboard for displaying results
- Notification and reminder system

2.3 Current System

Existing systems only perform text extraction or translation. They do not interpret document meaning or provide actionable insights.

3. REQUIREMENT ANALYSIS

3.1 Use Case Diagram(s)

(To be added)

3.2 Detailed Use Case

User uploads a document → system processes it → output is generated.

3.3 Functional Requirements

- Upload document
- Extract text using OCR
- Translate text
- Identify document type
- Extract key information
- Generate summary
- Provide action recommendations
- Display alerts

- Set reminders

3.4 Non-Functional Requirements

- High accuracy
- Fast response time
- Secure data handling
- User-friendly interface
- Scalability

4. DESIGN AND ARCHITECTURE

4.1 System Architecture

The system includes:

- Frontend (user interface)
- Backend (processing logic)
- OCR module
- NLP module
- Database

4.2 Data Representation

Input: Document (PDF/Image)
Output: Extracted text, summary, actions

4.3 Process Flow

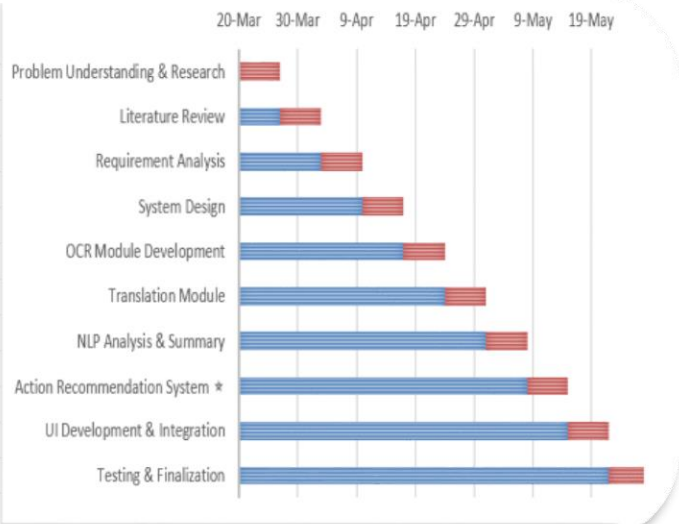
1. Upload document
2. Extract text
3. Translate text
4. Analyze content
5. Extract key data
6. Generate summary and actions
7. Display output

4.4 Design Models

Includes flowcharts, use case diagrams, and architecture diagrams.

4.4 Project Timeline (Gantt Chart)

Week	Task	Start Date	End Date
1	Problem Understanding & Research	20-Mar	26-Mar
2	Literature Review	27-Mar	2-Apr
3	Requirement Analysis	3-Apr	9-Apr
4	System Design	10-Apr	16-Apr
5	OCR Module Development	17-Apr	23-Apr
6	Translation Module	24-Apr	30-Apr
7	NLP Analysis & Summary	1-May	7-May
8	Action Recommendation System *	8-May	14-May
9	UI Development & Integration	15-May	21-May
10	Testing & Finalization	22-May	28-May



5. IMPLEMENTATION

5.1 Algorithm

1. Accept input document
2. Apply OCR
3. Detect language
4. Translate text
5. Extract keywords
6. Classify document
7. Generate summary
8. Provide actions

5.2 External APIs

- OCR API (text extraction)
- Translation API

5.3 User Interface

The system includes:

- Upload page
- Dashboard showing results
- Alerts and notifications

6. TESTING AND EVALUATION

6.1 Manual Testing

Manual testing is performed to verify the correctness of the system functions by interacting with the application directly. It ensures that the user interface works properly and that each module produces the expected output when valid inputs are provided.

6.1.1 System Testing

Once the system has been successfully developed, testing has to be performed to ensure that the system is working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing helps in finding hidden errors that may affect the user experience. The testing includes document upload, OCR extraction, translation, summary generation, action recommendation, and alert generation.

6.1.2 Unit Testing

Unit Testing 1: Upload Document

Testing Objective: To ensure that the document upload function is working correctly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify document upload after clicking the "Upload" button with a valid PDF file	File Type: PDF, File Name: visa_notice.pdf	The document is uploaded successfully and becomes available for processing	Pass
2.	Verify document upload after clicking the "Upload" button with a valid image file	File Type: JPG, File Name: insurance_bill.jpg	The image document is uploaded successfully and becomes available for processing	Pass

Unit Testing 2: OCR Text Extraction

Testing Objective: To ensure that the OCR module extracts text correctly from uploaded documents.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Extract text from a visa notice document	Input File: visa_notice.pdf	Text is correctly extracted from the document	Pass
2.	Extract text from a health insurance document image	Input File: insurance_bill.jpg	Text is correctly extracted from the image	Pass

Unit Testing 3: Translation Module

Testing Objective: To ensure that Korean text is translated correctly into English.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Translate extracted Korean visa text into English	Input Text: Korean visa notice	English translation is generated correctly	Pass
2.	Translate extracted Korean insurance text into English	Input Text: Korean insurance bill	English translation is generated correctly	Pass

Unit Testing 4: Summary Generation

Testing Objective: To ensure that the summary generation module is working properly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Generate summary for uploaded visa document	Document Type: Visa Notice	A clear and short summary is displayed	Pass
2.	Generate summary for uploaded tax document	Document Type: Tax Notice	A clear and short summary is displayed	Pass

6.1.3 Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each module is tested. This is to ensure that the system produced meets the specifications and requirements.

Functional Testing 1: Upload and Analyze Document

Objective: To ensure that the system correctly processes an uploaded document and displays the result.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Upload a visa document and run analysis	File Name: visa_notice.pdf	The system extracts text, translates it, and displays summary with action recommendations	Pass
2.	Upload an insurance bill and run analysis	File Name: insurance_bill.jpg	The system extracts text, translates it, and displays summary with payment deadline alert	Pass

Functional Testing 2: Display Action Recommendations

Objective: To ensure that the system shows appropriate actions based on the analyzed document.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Analyze visa renewal notice	Document Type: Visa	Action recommendation such as “Renew visa before deadline” is displayed	Pass
2.	Analyze insurance bill	Document Type: Insurance	Action recommendation such as “Pay insurance bill before due date” is displayed	Pass

Functional Testing 3: Alert Generation

Objective: To ensure that deadline and risk alerts are generated correctly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Analyze tax notice with near deadline	Deadline: 3 days remaining	Alert message is displayed to warn the user	Pass
2.	Analyze overdue insurance bill	Status: Overdue	Risk warning is displayed to the user	Pass

6.1.4 Integration Testing

Ensures all modules work together correctly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Upload document and run OCR processing	File Name: visa_notice.pdf	Document is uploaded successfully and text is extracted correctly	Pass
2.	Translate extracted text and perform analysis	Input Text: OCR output	Translated text is analyzed and summary is generated	Pass
3.	Generate summary and action recommendations	Document Type: Insurance Bill	Correct summary, deadline, and action recommendations are shown	Pass
4.	Display alerts after full document analysis	Deadline: Near due date	System shows proper alert and warning message	Pass

6.2 Automated Testing

Tools used:

Tool Name	Tool Description	Applied on [list of related test cases/FR/NFR]	Results
PyTest	Python testing framework used for backend module testing	OCR module, Translation module, Summary generation	Pass

Tool Name	Tool Description	Applied on [list of related test cases/FR/NFR]	Results
Selenium	Automation testing tool used for UI and workflow testing	Upload document, Analyze document, Display results	Pass

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The system helps foreign students understand official documents and take appropriate actions. It reduces confusion and prevents errors.

7.2 Future Work

- Mobile application
- Improved AI models
- Support for more document types
- Real-time notifications

8. REFERENCES

(Add research papers, APIs, websites later)